# Attention Is All You Need and the Craft of Prompt Engineering

How the Transformer architecture works, and how to steer it with language

A technical overview

***Abstract.*** *In 2017, Vaswani et al. introduced the Transformer, a sequence-transduction model built entirely on attention mechanisms and dispensing with recurrence and convolution. The architecture trained faster, parallelised better and scored higher on machine-translation benchmarks than the recurrent and convolutional systems that preceded it. It also became the foundation of today's large language models (LLMs). Because these models are controlled primarily through natural-language input, a practical discipline has grown around designing that input: prompt engineering. This paper reviews the Transformer's core ideas (scaled dot-product attention, multi-head attention, positional encoding), summarises its reported results, and then surveys prompting techniques such as few-shot prompting, chain-of-thought reasoning and self-consistency. We close by connecting the two halves: why attention makes prompts work, where it makes them fail, and how to iterate on prompts systematically.*

## 1. Introduction

Before 2017, state-of-the-art sequence models were mostly recurrent neural networks (RNNs), particularly LSTMs and GRUs, often paired with an attention layer. Recurrent models read a sentence one token at a time, carrying a hidden state forward. This sequential dependency has two costs: training cannot be parallelised across positions within an example, and information from distant tokens must survive many state updates, which makes long-range dependencies hard to learn.

The paper *Attention Is All You Need* (Vaswani et al., NeurIPS 2017) proposed a different design. Instead of passing a state through time, every token directly looks at every other token through an attention operation, and many such operations are stacked in layers. The result was a model that reached 28.4 BLEU on WMT 2014 English-to-German translation, improving over the best prior results (including ensembles) by more than 2 BLEU, while using a small fraction of the training cost.

The same architecture, scaled up and trained to predict the next token on vast text corpora, underlies GPT-style, Claude-style and other contemporary LLMs. These models do not expose a hundred tunable knobs; for most users the main interface is the **prompt**. Small changes in wording, ordering, examples or format can swing accuracy dramatically. Prompt engineering is the empirical practice of finding inputs that reliably elicit the behaviour we want.

This paper has two parts. Sections 2 to 4 describe the Transformer and its results. Sections 5 to 8 cover prompt engineering and its relationship to attention. Throughout, we include figures and tables to make the ideas concrete. Where a number is quoted from the literature we say so; where a figure is a teaching illustration rather than measured data we label it as such.

## 2. Background: From Recurrence to Attention

Attention was first used in neural machine translation to let a decoder look back at the encoder's states (Bahdanau et al., 2014) instead of compressing a whole sentence into one fixed vector. The key insight of the Transformer is that attention alone is sufficient: if every position can attend to every other position, we no longer need a recurrent state to move information around.

Table 1 compares the three layer families on the properties that matter most for sequence modelling. Self-attention connects any two positions in a constant number of sequential operations, which shortens the path that gradients and information must travel. The price is quadratic cost in the sequence length n, which is acceptable when n is smaller than the representation dimension d (true for most sentence-level translation) and is the reason long-context research is an active field today.

| Layer type | Complexity per layer | Sequential operations | Max. path length |
|---|---|---|---|
| Self-attention | $O(n^2 \cdot d)$ | O(1) | O(1) |
| Recurrent | $O(n \cdot d^2)$ | O(n) | O(n) |
| Convolutional | $O(k \cdot n \cdot d^2)$ | O(1) | $O(\log_k n)$ |
| Self-attention (restricted, r) | $O(r \cdot n \cdot d)$ | O(1) | O(n / r) |

*Table 1. Per-layer complexity, sequential operations and maximum path length (n = sequence length, d = representation dimension, k = kernel size, r = neighbourhood size). Adapted from Vaswani et al. (2017).*

## 3. The Transformer Architecture

The Transformer follows the familiar encoder-decoder layout. The **encoder** maps an input sequence of tokens to a sequence of continuous representations. The **decoder** then generates the output one token at a time, consuming its own previous outputs (auto-regression) and attending to the encoder's representations. Figure 1 shows the full stack. Both halves consist of N = 6 identical layers; each sub-layer is wrapped in a residual connection followed by layer normalisation.

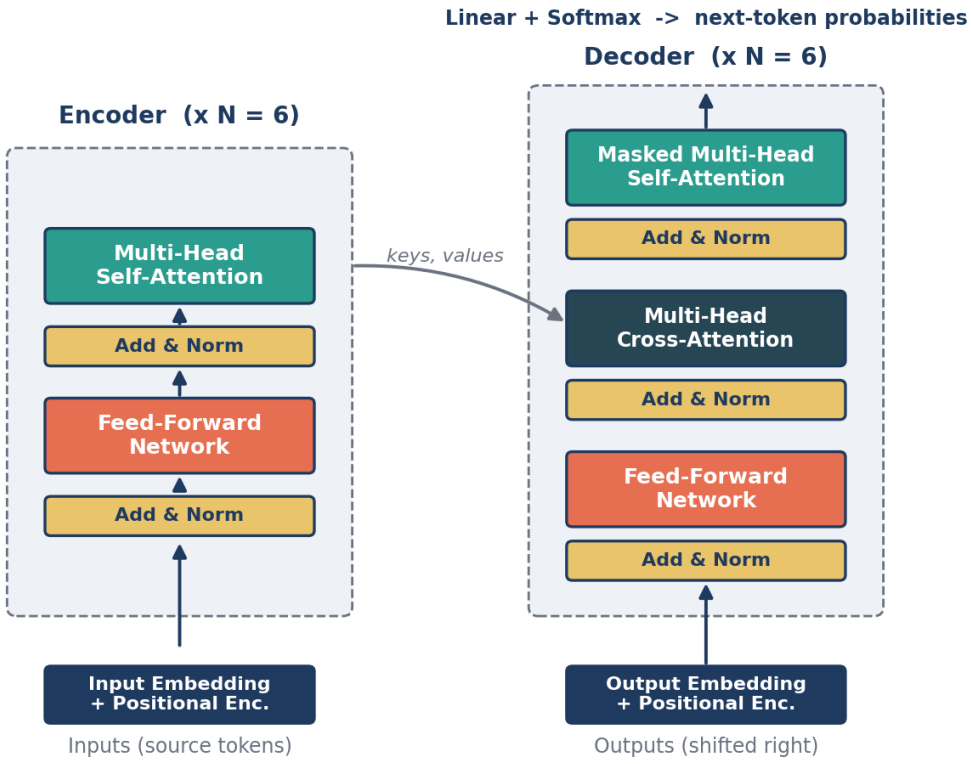



*Figure 1. The Transformer encoder-decoder architecture. The decoder has an extra cross-attention sub-layer that reads the encoder output, and its self-attention is masked so a position cannot see future tokens.*

### 3.1 Scaled Dot-Product Attention

Attention maps a *query* and a set of *key-value* pairs to an output. Each token is projected into a query vector Q (what am I looking for?), a key vector K (what do I contain?) and a value vector V (what do I contribute if selected?). The similarity of a query to every key is computed by a dot product, scaled, normalised with a softmax, and used to take a weighted average of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The division by the square root of $d_k$ matters. For large $d_k$, the dot products grow in magnitude and push the softmax into regions with extremely small gradients; scaling keeps the logits in a well-behaved range. In the decoder, a mask sets the logits for future positions to minus infinity before the softmax so that predictions for position i depend only on positions before i.

### 3.2 Multi-Head Attention

Rather than performing one attention operation over $d_{model}$-dimensional vectors, the Transformer projects Q, K and V h = 8 times with different learned linear maps, runs attention in parallel for each projection, concatenates the results and projects once more. Each head has dimension $d_k = d_{model} / h = 64$, so total cost is similar to single-head attention at full dimensionality. Different heads are free to specialise: in trained models, researchers have observed heads that track syntactic dependencies, heads that resolve coreference and heads that attend to the previous token.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

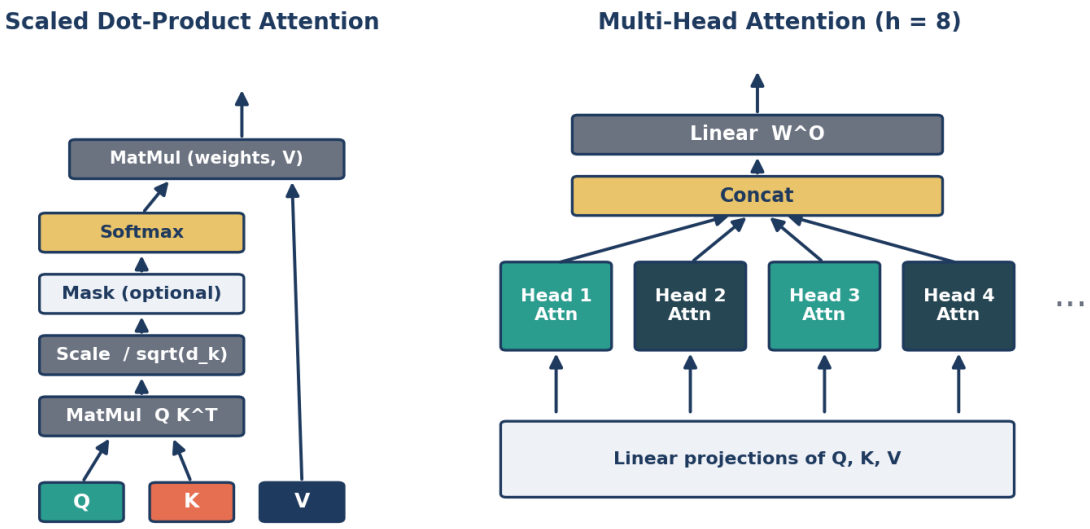



*Figure 2. Left: scaled dot-product attention. Right: multi-head attention runs several attention functions in parallel and merges them with a final linear projection.*

### 3.3 What Attention Looks Like

Figure 3 visualises a single attention head as a matrix: row i shows how much token i attends to every other token. The example mirrors the well-known coreference illustration in the original paper, where, in the sentence “The animal didn't cross the street because it was too tired”, the pronoun “it” should attend strongly to “animal”. **The weights here are hand-constructed for teaching purposes and are not output from a trained model.**

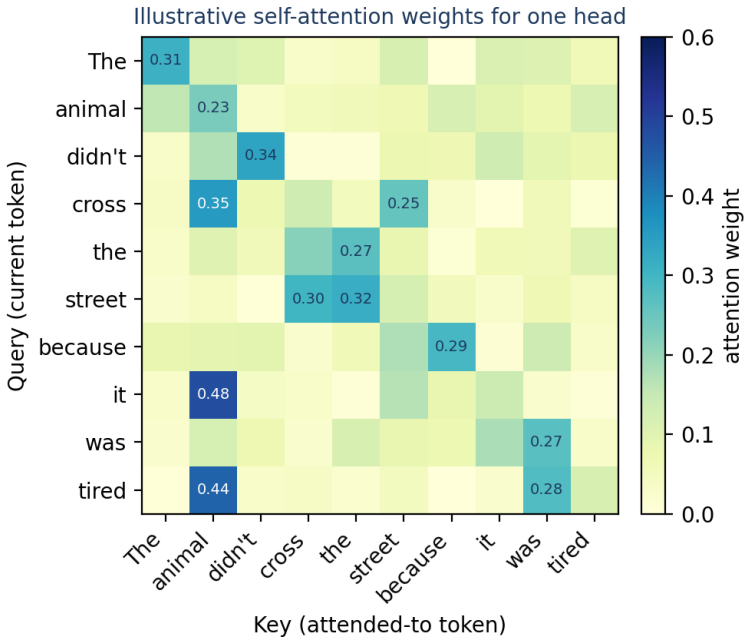



*Figure 3. Illustrative attention matrix. Each row sums to 1. The pronoun “it” places most of its weight on “animal”, resolving the reference in a single layer-wise step.*

### 3.4 Positional Encoding and the Feed-Forward Network

Attention is permutation-invariant: shuffle the input tokens and, without extra information, the outputs are shuffled in the same way. To inject word order, the Transformer adds a *positional encoding* to each input embedding. The original model uses fixed sinusoids of geometrically increasing wavelengths:

$$PE_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Each dimension therefore corresponds to a sinusoid, and the relative offset between two positions can be expressed as a linear function of their encodings, which makes it easy for the model to learn to attend by relative position. Figure 4 plots the encoding for d = 64.

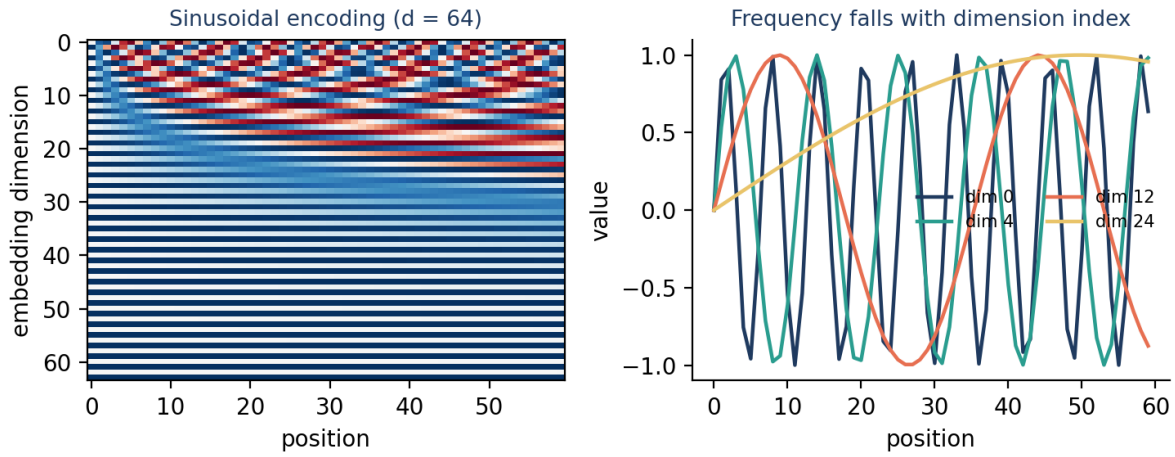



*Figure 4. Sinusoidal positional encodings. Low dimensions oscillate quickly; high dimensions vary slowly, giving each position a unique multi-scale signature.*

After attention, every position passes independently through the same two-layer feed-forward network. Together, attention (mixing information *across* positions) and the FFN (transforming information *within* a position) form the repeating unit.

$$\mathrm{FFN}(x) = \max(0,\, xW_1 + b_1)\, W_2 + b_2$$

## 4. Training Setup and Results

The original paper trained on the WMT 2014 English-German dataset (about 4.5 million sentence pairs, byte-pair encoding with a shared vocabulary of roughly 37,000 tokens) and the much larger English-French set (36 million sentences). Training used Adam with a warm-up learning-rate schedule, residual dropout and label smoothing. Table 2 lists the two configurations reported.

| Hyper-parameter | Base model | Big model |
|---|---|---|
| Layers (N) in encoder / decoder | 6 / 6 | 6 / 6 |
| Model dimension $d_{model}$ | 512 | 1024 |
| Feed-forward dimension $d_{ff}$ | 2048 | 4096 |
| Attention heads h | 8 | 16 |
| Per-head dimension $d_k = d_v$ | 64 | 64 |
| Dropout | 0.1 | 0.3 (EN-DE) |
| Parameters | ~65 million | ~213 million |

*Table 2. Transformer configurations (Vaswani et al., 2017).*

On translation quality, the big Transformer set a new state of the art on both language pairs (Figure 5 and Table 3). Perhaps more important than the BLEU gain was efficiency: because all positions are processed in parallel, the big model trained for roughly 3.5 days on eight GPUs, and the base model reached competitive quality at a fraction of the compute of earlier systems.

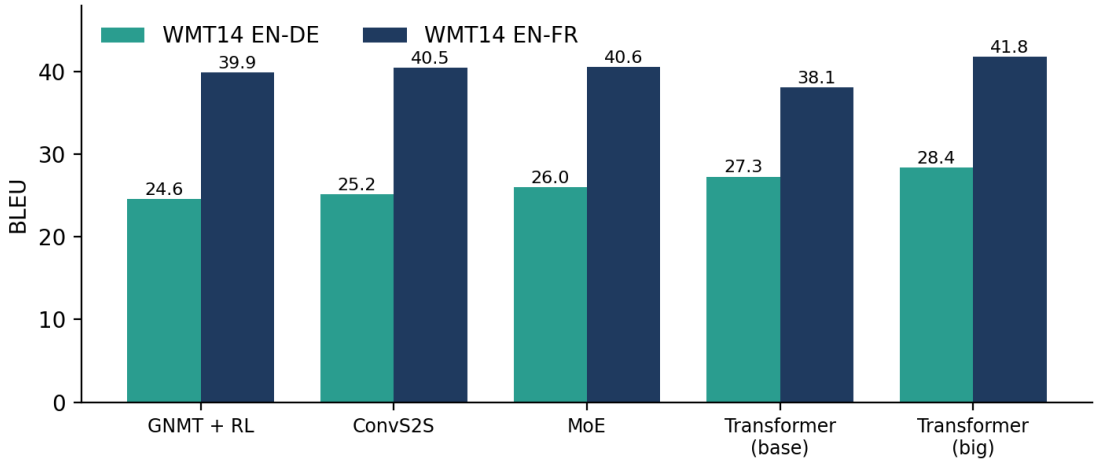



*Figure 5. BLEU scores on newstest2014 for representative systems reported in the paper. Higher is better.*

| Model | EN-DE BLEU | EN-FR BLEU | Training cost (FLOPs, EN-DE) |
|---|---|---|---|
| GNMT + RL | 24.6 | 39.92 | $2.3 \times 10^{19}$ |
| ConvS2S | 25.16 | 40.46 | $9.6 \times 10^{18}$ |
| MoE | 26.03 | 40.56 | $2.0 \times 10^{19}$ |
| Transformer (base) | 27.3 | 38.1 | $3.3 \times 10^{18}$ |
| **Transformer (big)** | **28.4** | **41.8** | $2.3 \times 10^{19}$ |

*Table 3. Translation quality and training cost. Single-model figures as reported by Vaswani et al. (2017); compute for the baselines is as estimated in that paper.*

# 5. Prompt Engineering: Fundamentals

A modern LLM is, at heart, a Transformer decoder trained to predict the next token. Everything the model "knows" about the task at inference time comes from two sources: the weights learned during training, and the **context window**, which is the text we place in front of it. Prompt engineering is the practice of designing that context so that the most likely continuation is the one we want.

## 5.1 Anatomy of a Prompt

Effective prompts are rarely a single sentence. They tend to combine a handful of components, summarised in Table 4. Not every prompt needs all of them, but making each component explicit removes ambiguity that the model would otherwise resolve by guessing.

| Component | Purpose | Example |
|---|---|---|
| Role / persona | Sets register, expertise and perspective | "You are a careful financial analyst." |
| Task instruction | States exactly what to do | "Summarise the report in 3 bullet points." |
| Context / data | Supplies the facts the answer must rest on | Pasted report text, retrieved documents |
| Examples (shots) | Demonstrates the desired mapping | Two input-output pairs |
| Output format | Makes the result machine- or human-usable | "Return valid JSON with keys a, b." |
| Constraints | Bounds length, tone, scope, safety | "Under 80 words; no speculation." |

*Table 4. Common building blocks of a well-structured prompt.*

## 5.2 A Structured Template

The following skeleton illustrates how the components are typically arranged. Placing instructions before the data and delimiting each part clearly helps the model separate what to do from what to process.

```
ROLE: You are a meticulous technical editor.
TASK: Rewrite the text below for a non-expert audience.
CONSTRAINTS: Max 120 words. Keep all numbers unchanged.
FORMAT: One paragraph, then a line starting 'Key takeaway:'.
TEXT: """ {{document}} """
```

## 5.3 Principles That Generalise

- **Be specific.** State audience, length, format and success criteria rather than relying on the model to infer them.
- **Show, do not only tell.** One or two representative examples often communicate a format better than a paragraph of rules.
- **Give the model room to think.** For multi-step problems, ask for intermediate reasoning before the final answer.
- **Separate instructions from data.** Use delimiters or tags so that text to be processed is not mistaken for instructions.
- **Say what to do, not only what to avoid.** Positive instructions are followed more reliably than long lists of prohibitions.
- **Test, do not assume.** A prompt is a hypothesis; evaluate it on realistic inputs, including awkward edge cases.

## 6. Core Prompting Techniques

**Zero-shot prompting** gives only an instruction. It works surprisingly well for tasks that resemble the model's training distribution. **Few-shot prompting**, popularised by Brown et al. (2020) with GPT-3, adds a small number of worked examples to the prompt; the model infers the task from the pattern without any weight updates, a behaviour called *in-context learning*.

**Chain-of-thought (CoT) prompting** (Wei et al., 2022) includes worked examples whose answers contain explicit intermediate reasoning steps. On arithmetic word problems, this raised the accuracy of the 540-billion-parameter PaLM model on GSM8K from 17.9% with standard few-shot prompting to 56.9%. Kojima et al. (2022) showed that even appending the phrase “Let's think step by step”, with no examples at all, lifts a large model on MultiArith from 17.7% to 78.7%. **Self-consistency** (Wang et al., 2022) samples several reasoning paths and takes a majority vote on the final answer, pushing GSM8K to 74.4% in the same PaLM setting. Figure 6 summarises these reported results.

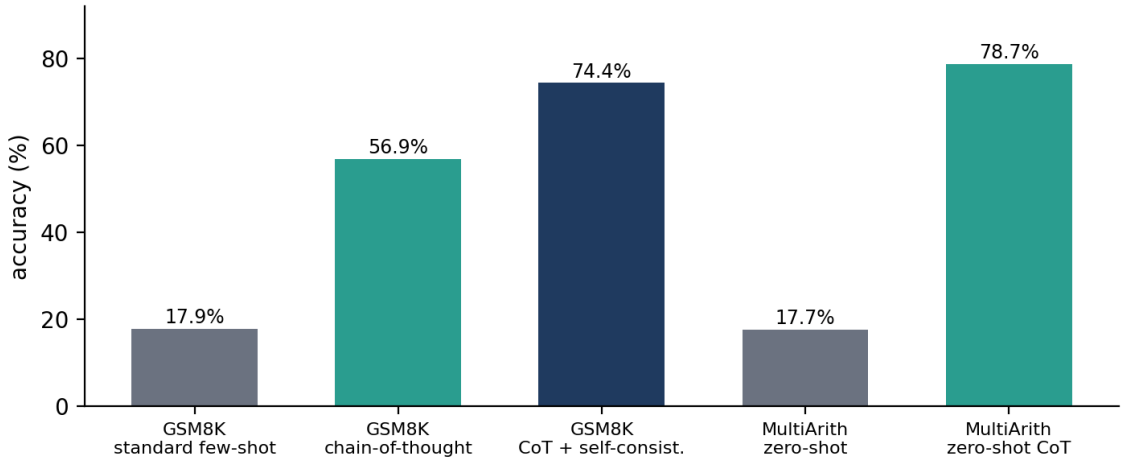



*Figure 6. Reported accuracy gains from reasoning-oriented prompting (Wei et al., 2022; Kojima et al., 2022; Wang et al., 2022). Numbers apply to the specific models and benchmarks in those papers and do not transfer automatically.*

Table 5 compares the main techniques on cost and typical use. Cost is dominated by output length and the number of calls: CoT lengthens each response, while self-consistency multiplies it by the number of sampled paths.

| Technique | Idea | Relative cost | Best for |
|---|---|---|---|
| Zero-shot | Instruction only | Lowest | Simple, common tasks |
| Few-shot | Add k worked examples | Low-medium | Format-sensitive tasks, classification |
| Chain-of-thought | Elicit step-by-step reasoning | Medium | Maths, logic, multi-step problems |
| Self-consistency | Sample many CoT paths, vote | High | Tasks with a checkable final answer |
| Retrieval-augmented | Insert retrieved documents | Medium | Fresh or private knowledge |
| Tool / agentic use | Model calls tools in a loop | Variable | Actions, calculation, search |

*Table 5. Prompting techniques at a glance.*

# 7. Where the Two Halves Meet: Attention and Prompts

Why does rewording a prompt change behaviour so much? Part of the answer lies in the mechanism of Section 3. In each layer, every generated token computes attention weights over *all* tokens in the context, including the prompt. Whatever the prompt contains therefore competes for attention on every step, and the words that end up with high weight shape the next-token distribution. This mechanistic picture is a useful intuition, though attention weights alone are not a complete explanation of model behaviour.

## 7.1 Practical Consequences

- **Examples act as retrieval cues.** Few-shot demonstrations give the model tokens to match against; closely similar examples tend to be more helpful than generic ones.
- **Position matters.** Empirical work (Liu et al., 2023) found that models often use information at the beginning and end of a long context better than information buried in the middle. Putting key instructions or evidence at the edges is a cheap mitigation.
- **Reasoning tokens are working memory.** The decoder has no hidden scratchpad besides the tokens it has written. Chain-of-thought lets later steps attend to earlier intermediate results, which is a plausible reason it helps on multi-step problems.
- **Context is finite and costly.** Because attention scales quadratically with length, long prompts raise latency and cost, and irrelevant text can dilute focus. Include what is needed, and no more.
- **Delimiters create structure.** Tags, headings and quotes give attention clear boundaries, reducing the chance that data is read as instructions.

## 7.2 A Repeatable Workflow

Prompt design works best as an experimental loop (Figure 7) rather than a one-off act of inspiration. Define what "good" means, ideally with a small labelled test set; draft a prompt; measure; read the failures; change one thing at a time; and keep versions so that regressions can be traced.

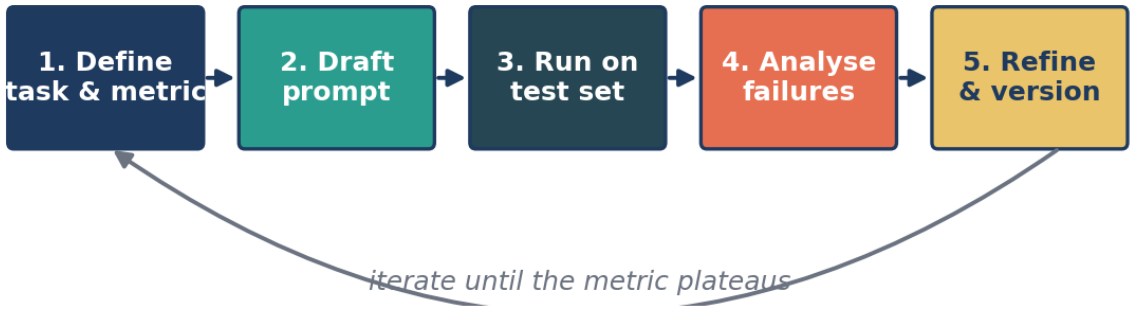



*Figure 7. An iterative prompt-development loop. Changing one variable per iteration keeps results interpretable.*

| Symptom | Likely cause | Try this |
|---|---|---|
| Ignores format | Format only implied | State format explicitly; add one example |
| Wrong on multi-step tasks | No room to reason | Ask for steps first, answer last |
| Inconsistent between runs | High sampling variance | Lower temperature; use self-consistency |
| Drifts off topic in long input | Key instruction buried | Repeat instruction at the end; trim context |
| Invents facts | No grounding in context | Supply sources; permit "I don't know" |

*Table 6. A short troubleshooting guide for common prompt failures.*

## 8. Limitations and Risks

Both the architecture and the practice have important limits. On the architecture side, quadratic attention cost restricts context length and has motivated sparse, linear and memory-efficient variants. On the practice side, prompts are brittle: a technique that works for one model may fail on another, and reported gains depend on the benchmark and model scale. Prompt engineering also cannot supply knowledge a model lacks, nor guarantee truthfulness; grounding in trusted sources and verification of outputs remain necessary.

Security is a further concern. When prompts include untrusted text (web pages, emails, user uploads), that text may contain instructions that try to override the developer's intent, an attack known as *prompt injection*. Mitigations include separating trusted and untrusted content, limiting tool permissions, validating outputs and keeping a human in the loop for consequential actions.

## 9. Conclusion

The Transformer showed that attention, applied at every layer and across all positions, is a sufficient and highly parallelisable basis for sequence modelling. Its descendants turned natural language into a programming interface, and prompt engineering emerged as the discipline of using it well. Understanding the first helps with the second: knowing that every token attends to the whole context explains why clarity, structure, ordering and relevant examples matter. As models improve, specific tricks will come and go, but the underlying habits of stating goals precisely, supplying the right context, measuring results and iterating will remain useful.

## References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). *Attention Is All You Need.* Advances in Neural Information Processing Systems 30.

[2] Bahdanau, D., Cho, K., Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate.* arXiv:1409.0473.

[3] Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners.* Advances in Neural Information Processing Systems 33.

[4] Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS 2022.

[5] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., Iwasawa, Y. (2022). *Large Language Models are Zero-Shot Reasoners.* NeurIPS 2022.

[6] Wang, X., et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* arXiv:2203.11171.

[7] Liu, N. F., et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* arXiv:2307.03172.

**Note on data:** Benchmark numbers are quoted from the cited papers. Figure 3 is a hand-built illustration; Figure 4 is computed from the positional-encoding formula; all other diagrams are schematic.